

Capability-Oriented Software Engineering (COSE): An Incremental Methodology for Building Reusable Capability Libraries

Enrique Solis Carvajal

Abstract—Software development frequently reconstructs functionality already implemented in previous projects because reusable knowledge remains coupled to application-specific code and execution context. This paper presents Capability-Oriented Software Engineering (COSE), an incremental methodology that treats a cumulative library of validated capabilities as a principal organizational engineering asset. COSE follows a reuse-first resolution cycle: define a functional objective, discover and compose existing capabilities, identify the unresolved functional residual, and develop only the missing responsibilities. Capabilities encapsulate independently testable functions, while project-specific bindings are supplied externally through execution maps. Deterministic processing and bounded cognitive processing are separated, with large language models accessed through replaceable interfaces. Persistent artifacts support inspection, provenance, and reconstruction of execution state. The architecture additionally distinguishes contextual authorization, capability messaging, operation-level governance, and prospective task-state persistence. Evidence is drawn from seven software systems, an open-ended autonomy case study, and an additional practice report concerning municipal electricity-billing automation. Reported observations include completion of a 36-configuration trading-infrastructure test, approximately eight days of audited live-paper operation, structural validation of a generated crawler, model substitution without upstream parser changes, and governed acceptance and rejection of execution requests. These observations support architectural feasibility and identify concrete failure modes, but do not establish causal reductions in engineering cost, general reliability guarantees, or autonomous profitability. The paper consolidates the methodology, qualifies its evidence, and identifies measurements required for independent evaluation.

Index Terms—Software Engineering Methodology, Capability Reuse, Residual Problem Decomposition, Contextual Binding, Capability Libraries, Large Language Models, Artifact-Based Verification, Governed Execution, Task-State Persistence.



1 INTRODUCTION

SOFTWARE engineering paradigms—including structured programming, object-oriented design, service-oriented architecture, microservices, and multi-agent systems—provide mechanisms for managing complexity and promoting reuse. Nevertheless, functional knowledge often remains embedded in project-specific implementations, organizational conventions, and tacit developer experience.

COSE emerged inductively from engineering practice rather than from a prior formal theory. Early experiments organized work around multi-agent “characters” with broad collections of skills. During the development of Dol-Goldur, a catalog-enrichment and publication system, this arrangement was replaced by explicit pipelines of smaller functional units. CatalogVision and Andigraf subsequently reinforced the observation that narrowly scoped capabilities were easier to inspect, adapt through composition, and reuse than broad agent identities.

The methodology was later applied to Website Intelligence Factory, Knowledge Factory, Mammon, and a multi-

model middleware infrastructure.

The initial formalization emphasized problem decomposition and cumulative capability registration. Subsequent methodological consolidation clarified an operational ordering already present in practice:

Existing capabilities should be considered before new responsibilities are decomposed and implemented.

The objective is therefore not to maximize the number of capabilities. It is to resolve a problem while minimizing unnecessary reconstruction and preserving coherent functional boundaries.

1.1 Core Tenets

COSE rests on four consolidated tenets:

- 1) **Reuse-First Resolution:** Search the existing Capability Library and attempt composition before creating new functionality.
- 2) **Residual-Driven Development:** Decompose and implement only the functional responsibilities that remain inadequately covered.
- 3) **Context-Independent Capabilities:** Encapsulate functional knowledge within capabilities and provide project-specific execution context externally.
- 4) **Cumulative Composition with Persistent Evidence:** Register validated capabilities for future use and execute compositions through inspectable, persistent artifacts.

• E. Solis Carvajal is with Change Tensors Solutions — Systems Architecture & Engineering, Santiago, Chile (e-mail: esolis@tcgplanet.cl).
 • Research conducted within the COSE Multi-Agent and Systems Architecture Infrastructure.
 • DOI: 10.5281/zenodo.22029421. Canonical Repository: github.com/Solis666/cose-methodology
 Manuscript version 5.1, October 2026. Version history: v1.0 (June 2026), v2.1 (August 2026), v3.0 (September 2026), v3.1 (October 2026).

The methodology does not require a Large Language Model (LLM). Entirely deterministic systems can satisfy its principles. When cognitive processing is useful, it is implemented as a bounded capability behind a stable interface.

The consolidated lifecycle follows:

Problem → *Discovery* → *Reuse* → *Composition* → *Residual Evaluation* → *Residual Decomposition* → *Minimal New Development* → *Validation* → *Registration* → *Contextual Binding* → *Execution*.

1.2 Evidence-Based Reconstruction

A recurring observation in the engineering ecosystem was disagreement between documentation and execution evidence. Some components had advanced beyond their recorded status; others were operating without corresponding documentation updates.

COSE therefore adopts an evidence-priority rule:

Operational state must be reconstructed from attributable execution evidence rather than inferred from documentation alone.

The earlier expression, “The log is the plan; the filesystem is reality,” captures the motivation but requires qualification. Filesystems can contain stale, incomplete, duplicated, or incorrectly attributed artifacts. Evidence does not become authoritative merely because it exists on disk.

A reconstruction should establish:

- 1) The identity and version of the component.
- 2) The execution or observation to which an artifact belongs.
- 3) The inputs, outputs, timestamps, and available provenance.
- 4) The validation actually performed.
- 5) The limits of the resulting claim.

A generated crawler with a passing structural report establishes something different from a crawler that successfully extracts authorized live-site data. Similarly, a shared log containing several runs cannot establish the boundary of one run without additional evidence.

1.3 Contributions and Status of the Consolidation

This consolidated revision contributes:

- 1) A reuse-first resolution cycle based on functional residuals.
- 2) A stopping criterion for capability decomposition.
- 3) An explicit separation between functional knowledge and execution context.
- 4) Definitions of the Capability Detector, execution map, Map Normalizer, and Map Router.
- 5) A separation of contextual authorization, message transport, and capability-level enforcement.
- 6) A qualified formalization of cumulative library growth and model substitution.
- 7) Integration of the existing evidence-reconstruction, messaging, and task-state mechanisms.
- 8) An expanded interpretation of Andigraf and a separate municipal-billing practice report.

- 9) Revised empirical claims that distinguish observations, design requirements, and untested hypotheses.

Residual decomposition, contextual independence, and execution-map governance are presented as **practice-derived formalizations**. They are not represented as terminology that necessarily appeared explicitly in Version 2.1.

2 RELATED WORK

2.1 Multi-Agent Systems and Agentic Frameworks

Multi-agent systems provide foundations for cooperation, coordination, and distributed task allocation [1]. ReAct [2] and AutoGen [3] demonstrate important forms of language-model reasoning, tool use, and multi-agent interaction. COSE addresses a related but distinct organizational question: how should functional implementations become reusable assets across projects?

2.2 Tool-Augmented Language Models

Toolformer [4] and ToolLLM [5] investigate how language models can select and use external tools. COSE instead emphasizes the engineering boundary around those tools: their contracts, execution permissions, contextual bindings, and evidence.

2.3 Reusable Components and Compositional Architectures

Microservices [6] promote independently managed service boundaries, while the Unix philosophy [7] emphasizes focused, composable programs. The open/closed principle [9] treats a validated unit as closed to modification once in use. COSE combines reuse-first resolution, residual decomposition, and project-context externalization with these principles.

2.4 Deterministic–Cognitive Separation

Kautz [8] discusses integration of neural and symbolic reasoning. COSE’s narrower architectural claim is that predictable transformations and policy enforcement should not depend unnecessarily on probabilistic language-model behavior.

2.5 Persistent Obligations and Verification

COSE distinguishes prospective obligation state from retrospective execution records, emphasizing active representation of pending tasks when conditions are met.

3 THEORETICAL FRAMEWORK & NOTATION

3.1 Formal Definitions

- **Problem (*P*)**: Functional requirement producing operational value.
- **Micro-problem (*M*)**: Distinguishable functional responsibility with input/output contract.
- **Capability (*C*)**: Autonomous, reusable functional unit resolving one micro-problem.

- **Capability Library ($\mathcal{L}$):** Cumulative collection of validated capabilities.
- **Eligible Capability Set ($\mathcal{E}_t$):** Subset of library currently permitted ($\mathcal{E}_t \subseteq \mathcal{L}_t$).
- **Pipeline (Π):** Composition of capability invocations.
- **Artifact (T):** Persistent data structure carrying inputs/outputs/state.
- **Capability Detector (D):** Discovery mechanism retrieving candidates.
- **Functional Residual (R):** Unsatisfied requirements under evaluation procedure.
- **Execution Map (X):** External declaration of authorized bindings.
- **Task-State Artifact (S):** Record of unresolved obligations.

3.2 Reuse-First Resolution and Residual Decomposition

Given problem P , the detector retrieves candidate capabilities:

$$K_0 = D(P, \mathcal{E}_t) \quad (1)$$

An initial composition is constructed: $\Pi_0 = \text{compose}(K_0)$.

Let $Q(P)$ be declared obligations and $\text{Sat}(\Pi_i, E_i)$ be obligations satisfied under evidence E_i . The functional residual is:

$$R_i = Q(P) \setminus \text{Sat}(\Pi_i, E_i) \quad (2)$$

Only unresolved responsibilities are decomposed:

$$\text{decompose}(R_i) \rightarrow \{M_{i,1}, M_{i,2}, \dots, M_{i,n_i}\} \quad (3)$$

Each micro-problem is checked against the library:

$$\text{resolve}(M) = \begin{cases} C_k \in \mathcal{E}_t, & \text{if a suitable capability exists,} \\ \text{develop}(C_{\text{new}}), & \text{otherwise.} \end{cases} \quad (4)$$

Library cumulative growth satisfies:

$$|\mathcal{L}_{t+1}| \geq |\mathcal{L}_t| \quad (5)$$

3.3 Atomicity and Decomposition Stopping Rule

Decomposition stops when a capability: 1) Has one independently describable responsibility; 2) Has observable inputs/outputs; 3) Can be tested independently; 4) Can be separated from the parent objective; 5) Can be reused via contextual rebinding.

Parameterization Heuristic: Does the new case change a value consumed by the existing procedure (parameterize), or does it require a different procedure (develop separate capability)?

3.4 Functional Knowledge vs. Contextual Binding

Capabilities hold functional knowledge; execution maps supply contextual information.

$$T_{\text{out}} = C(T_{\text{in}}; b_X) \quad (6)$$

TABLE 1
Functional Knowledge vs. Execution Context

Inside Capability	Supplied Externally
Functional responsibility	Project identity
Transformation rules	Concrete input/output locations
Input/output schema	Pipeline file names
Intrinsic domain logic	Invocation ordering
Precondition validation	Run identifier & approved bindings

3.5 Persistent Task State

Task state $S_t = \{s_1, s_2, \dots, s_m\}$ uses entry tuple $s = (id, \sigma, \rho, \varepsilon, \omega)$ where σ is state, ρ is re-emergence predicate, ε is evidence reference, and ω is responsible role.

Active re-emergence requires:

$$\rho(s, t) = \text{true} \Rightarrow \text{enqueue}(\text{reviewRequest}(s)) \quad (7)$$

4 ARCHITECTURAL PARADIGM

4.1 Core Design Principles

- 1) **Single Responsibility & Atomic Granularity**
- 2) **Deterministic-Cognitive Segregation**
- 3) **Artifact-Based State Persistence**
- 4) **LLM as a Bounded Component**
- 5) **Model Independence & Interchangeability**
- 6) **Task-State Persistence Across Invocations**

4.2 Execution Governance & Messaging Bus

An Execution Map declares execution context:

$$X = (\text{pipeline_id}, \text{cap_refs}, \text{bindings}, \text{deps}, \text{auth_constraints}) \quad (8)$$

Request execution requires all policy checks to pass:

$$\text{PermitExecute}(r) = \text{AuthorizedCaller}(r) \wedge \text{AllowedByMap}(r) \wedge \text{ValidContract}(r) \wedge \text{AllowedByPolicy}(r) \quad (9)$$

5 EMPIRICAL EVALUATION & CASE STUDIES

5.1 Case Study 1 — Mammon: Algorithmic Trading Platform

Mammon separates market acquisition, candle validation, signal generation, paper execution, and simulation.

TABLE 2
Mammon Stress-Matrix Results

Asset	Timeframes	Windows	Completion	Failures
BTCUSDT	15m, 1h, 4h	200, 500, 1000	100%	0
ETHUSDT	15m, 1h, 4h	200, 500, 1000	100%	0
SOLUSDT	15m, 1h, 4h	200, 500, 1000	100%	0
BNBUSDT	15m, 1h, 4h	200, 500, 1000	100%	0

TABLE 3
Mammon Continuous Live-Paper Audit

Metric	Reconstructed Value
Symbol / Timeframe	ETHUSDT / 1h
Observation Window	2026-09-03 17:00 to 2026-09-11 19:00 UTC
Attributed Blocks	177 (8 days)
Observed Cadence	91% hourly
Initial Equity	\$10,000.00
Final Equity	\$9,592.82 (-\$407.18 / -4.07%)
Realized Profit Accumulator	-\$298.05 (Unreconciled diff: \$109.13)
Termination	Host OS restart

5.2 Knowledge Factory, WIF, and Governed Middleware

Knowledge Factory demonstrated model provider substitution without upstream code modification. Website Intelligence Factory (WIF) generated a structurally verified crawler despite target HTTP 403 blocks. Middleware testing confirmed exact execution refusals for unauthorized directories and commands.

5.3 Business Applications & Municipal Billing

Andigraf demonstrated residual-driven adaptation by introducing a separate capability for gaming notebook classification rather than bloating existing product-matching routines. The Municipal Billing report automated 928 CGE customer numbers where authentication persistence was the primary residual.

TABLE 4
Reported Internal Conformity Assessment

Component	P1	P2	P3	P4	P5	P6
WIF	✓	✓	✓	✓	✓	×
Monolithic Bridge	×	~	✓	×	~	×
Mammon	✓	✓	✓	✓	✓	×
Bounties / GitHub	✓	~	✓	~	✓	~
Bridge UI	×	~	~	×	×	×
Messaging Bus	✓	✓	✓	✓	✓	~

6 DISCUSSION & FUTURE WORK

The economic hypothesis posits that effort $E(P) = E_{\text{discovery}} + E_{\text{composition}} + E_{\text{new}} + E_{\text{validation}} + E_{\text{governance}}$ is reduced over time as E_{new} declines. Future work requires independent cross-project metrics, formal capability contracts, and automated task re-emergence validation.

7 CONCLUSION

COSE establishes an operational methodology for treating functional software components as cumulative organizational assets, separating functional logic from project-specific execution context and establishing inspectable, evidence-based execution boundaries.

APPENDIX

Repository: <https://github.com/Solis666/cose-methodology>. DOI: 10.5281/zenodo.22029421.

Copy-Validate-Deprecate pattern isolates replacement modules while preserving backward compatibility through shims.

Maintains unresolved obligations with fields: TASK_ID, RESPONSIBLE_PARTY, STATE, REMAINING_OBLIGATION, RE_EMERGENCE_CONDITION, and EVIDENCE.

REFERENCES

- [1] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.
- [2] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.
- [3] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv:2308.08155*, 2023.
- [4] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.
- [5] Y. Qin *et al.*, "ToolLLM: Facilitating large language models to master 16,000+ real-world APIs," in *Proc. ICLR*, 2024.
- [6] M. Fowler and J. Lewis, "Microservices: A definition of this new architectural term," 2014.
- [7] E. S. Raymond, *The Art of Unix Programming*. Addison-Wesley, 2003.
- [8] H. Kautz, "The third AI summer," *AI Magazine*, vol. 43, no. 1, 2022.
- [9] B. Meyer, *Object-Oriented Software Construction*. Prentice-Hall, 1988.